PERSONAL LEARNING PLAN

Applied AI Learning Roadmap

A practical 30-day path for a fullstack developer to learn applied AI by building NestJS + Next.js features with local Ollama models.

VERSION
1.0

DATE
August 2026

AUTHOR
Swapnil Sagvekar

AUDIENCE
Fullstack developer (NestJS + Next.js)

SCOPE
Learn applied AI by building a real chat product with local models (Ollama)

Goal

Become a **fullstack + AI engineer** who can ship AI features in real apps — not a research scientist training giant models from scratch.

By the end of this roadmap you will be able to:

- Call LLMs from NestJS (local Ollama or paid APIs)
- Stream responses to a Next.js UI
- Persist chat history in a database
- Build RAG over your own documents
- Add tool calling / simple agents
- Reason about cost, latency, evals, and safety

What you will learn (big picture)

Area	What it means in simple words
LLM APIs	Talk to an AI model from your backend
Prompts	Instructions that steer model behaviour
Streaming	Show answers token-by-token as they generate
Memory / history	Save chats so context continues across sessions
Embeddings	Turn text into numbers so similar meaning can be found
RAG	Answer questions using *your* docs, not only model memory
Tool calling	Let AI call your APIs / DB / actions
Evals	Check if answers are actually good
Production basics	Auth, rate limits, logging, cost control

Approach

1. **Build first** — learn by shipping features in `ai-chat-starter`
2. **Local first** — use Ollama (free) before paid OpenAI/Anthropic keys
3. **One skill per day/week** — don't jump to fine-tuning too early
4. **Same stack you already know** — NestJS + Next.js + Prisma

---

Progress so far

Day	Topic	Status	Practice outcome
Day 1	Local LLM chat API + UI	✅ Done	Nest `POST /chat` + Next chat page via Ollama
Day 2	Streaming replies	✅ Done	`POST /chat/stream` (SSE) + live typing UI
Day 3	Save conversations in DB	✅ Done	Prisma + SQLite; sidebar history load/delete
Day 8+	RAG over uploaded docs	✅ Done	Upload .txt/.md → chunk → embed → retrieve → answer with sources
Day 15+	Tool calling / agents	✅ Started	Tools: list docs, count chats, search docs, server time

Practice project path: `~/Desktop/Swapnil/projects/ai-chat-starter`

30-day learning plan

Week 1 — Foundations (chat product)

Day	Learn	Build
1	Tokens, prompts, system vs user messages	Local chat API + UI
2	Streaming / SSE	Token-by-token replies
3	Persist threads	Conversation + Message tables
4	Better UX	Titles, timestamps, clear chat, error states
5	Env switching	Same code works for Ollama **or** OpenAI
6–7	Review + notes	Write short “what I learned” notes

Week 1 outcome: a durable local chat app you can demo.

Week 2 — RAG (most important skill)

Day	Learn	Build
8	What RAG is (retrieve → stuff into prompt → answer)	Concept notes
9	Chunking PDFs / markdown	Upload + split docs
10	Embeddings	Store vectors (pgvector or local vector store)
11	Retrieval	Top-k similar chunks for a question
12	Cite sources	Show which doc chunk was used
13–14	Polish	"Ask my docs" end-to-end demo

Week 2 outcome: AI answers only from *your* documents.

Week 3 — Tools / agents

Day	Learn	Build
15	Function / tool calling	Model requests a tool with JSON args
16	Structured output	Force JSON responses
17	Simple agent loop	think → call tool → answer
18	App actions	e.g. create note, query DB, summarize week
19–21	Mini agent feature	One useful action wired into UI

Week 3 outcome: AI that can *do* things, not only chat.

Week 4 — Production quality

Day	Learn	Build
22	Evals	Manual test set of Q&A; score answers
23	Cost & latency	Log tokens; compare models
24	Safety basics	Prompt injection awareness; input limits
25	Auth	Protect chat routes
26	Observability	Request logs / traces
27–30	Portfolio polish	README, demo video notes, architecture diagram

Week 4 outcome: interview-ready applied AI demo.

Skills order (do not skip)

1. Prompting + API usage
2. Streaming
3. Persistence
4. **RAG**
5. Tool calling
6. Evals + cost/latency
7. Light classic ML theory (optional — Iris-style)
8. Fine-tuning (only if RAG is not enough)

Stack for this roadmap

Layer	Choice	Why
Backend	NestJS	Matches your current work
Frontend	Next.js	Matches your current work
Local model	Ollama (`llama3.2:3b`)	Free, private, good for learning
Paid models (later)	OpenAI / Anthropic	Higher quality when needed
DB	SQLite → Postgres later	Start simple
ORM	Prisma	Fast schema + migrations
Vectors (Week 2)	pgvector or Chroma	RAG storage

Keep code configurable:

```
LLM_BASE_URL=http://localhost:11434/v1
LLM_API_KEY=ollama
LLM_MODEL=llama3.2:3b
```

Later, switch to a paid provider by changing env only.

What you will NOT focus on yet

- Training GPT-scale models from scratch
- Buying GPUs / CUDA clusters
- Rust / Solana / blockchain (separate path)

- Every new AI wrapper library
- Kaggle competitions as the main path

Those can come later. Applied AI shipping skills first.

How to study each day (60–90 min)

1. **20 min** — read/watch one concept
2. **40–60 min** — code in `ai-chat-starter`
3. **5 min** — write 3 lines: what broke, what you fixed, what you learned

Consistency beats long weekend marathons.

Portfolio projects (target)

#	Project	Proves you can…
1	Local AI chat with streaming + history	LLM API + product UX
2	Doc Q&A (RAG)	Ground answers in private data
3	Internal agent (tool calling)	AI that takes actions
4	AI feature inside a real Skitii-style app	Production judgment

Three strong projects beat twenty unfinished tutorials.

Checklist — "I understand this"

After each milestone, you should explain in your own words:

- [ ] What is a system prompt?
- [ ] Why send full message history?
- [ ] What is a token?
- [ ] Why stream responses?
- [ ] Why save conversations in DB?
- [ ] What problem does RAG solve?
- [ ] When is tool calling better than plain chat?
- [ ] How do you know the answer is good (evals)?

Immediate next step

Week 2 / Day 8 — start RAG

1. Add document upload to `ai-chat-starter`
2. Chunk text
3. Embed + store
4. Retrieve relevant chunks for each question
5. Ask the model to answer using only those chunks

Success definition

You are successful when you can:

1. Explain what your AI feature does and why
2. Build LLM + RAG features without copying blindly
3. Debug bad answers (prompt, retrieval, or data)
4. Demo a working app to a teammate or interviewer

Notes

- Prefer **understanding the raw API** before heavy frameworks (LangChain, etc.).
- Data quality and clear prompts matter more than fancy model names.
- Local models are weaker than GPT-4/Claude — that is fine while learning architecture.